DA2304 : Assignment 3

Aakash DA24B028

Integration and Verification

The assignment asks us to show that the `.vm` files produced by our compiler run correctly on the Hack architecture.

Why we used the VM Emulator

When all `.vm` files (our code plus the Jack OS files like `Math.vm`, `Memory.vm`, `Output.vm` etc.) are fed into a VM Translator and converted to a single `.asm` file, the total comes to around 37,000 lines of assembly. The Hack CPU has a hard limit of 32,768 instructions, so loading this `.asm` into the CPU Emulator causes it to fail.

The NAND2Tetris Web IDE VM Emulator does not have this problem because it runs `.vm` files directly without converting to assembly. It has the Jack OS built in, so we only needed to load our two files:

```
Conv.vm
Main.vm
```

Result

We loaded both files into the Web IDE VM Emulator and ran at full speed. The screen showed:

```
5x5 Convolution output:
30 30 30
30 30 30
30 30 30
9x9 Convolution output:
54 54 54 54 54 54 54
54 54 54 54 54 54 54
54 54 54 54 54 54 54
54 54 54 54 54 54 54
54 54 54 54 54 54 54
54 54 54 54 54 54 54
54 54 54 54 54 54 54
```

The program halted cleanly with no errors. The emulator screenshot is included in the submission as proof.

This confirms the full pipeline works correctly:

`Conv.jack` → `Conv.vm` → VM Emulator → Correct output on screen

Design Justification for Conv.jack

The assignment says we can add extra fields, static variables, methods, or helper functions beyond the skeleton, as long as we justify them.

We did **not add any extra fields, static variables, or methods**. Our `Conv` class has exactly the same structure as the skeleton:

- `static Array ArrayFilter` – same as skeleton
- `static int stride` – same as skeleton
- `field Array ArrayInput` – same as skeleton
- `field Array ArrayOutput` – same as skeleton
- `field int ArraySize` – same as skeleton

And the same 7 methods: `new`, `initFilter`, `getArrayInput`, `calcOutputDim`, `conv2d`, `printOutput`, `dispose`.

However, inside these methods we made some implementation choices that are worth explaining:

1. **Local variables in constructor** : We added two local variables `outDim` and `outTotal` inside `new`. These are needed to calculate how big the output array should be before calling `Array.new`. Without them we cannot allocate the right amount of memory for `ArrayOutput`.

2. **Bounds checking in conv2d** : The assignment requires bounds checking. We check if `outDim < 1` which means the input is too small for even one window. If this happens we write `-1` to `RAM[0]` using `Memory.poke(0, -1)` as an error signal, which follows the same convention used in Assignment 2.

3. **calcOutputDim called from constructor and conv2d** : We call `calcOutputDim()` in two places: once in the constructor to allocate `ArrayOutput`, and once at the start of `conv2d` to know how many iterations to do. This avoids repeating the formula in multiple places.

4. **Stride set to 1** : The assignment says stride can be 1 or 2. We set it to 1 in the constructor because it gives a larger output (3×3 for 5x5 input) which is easier to verify.

Exercise 3 – Analysis Questions

Q1 : Tokenizer Design

Problem with simple regex

If we use a regex to remove comments, it only looks at one line at a time. A block comment like `/* ... */` can go across many lines. So the regex will not know that line 3 is still inside a comment that started on line 1.

Example where it goes wrong

```
1 var int x;
2 /* This comment
3    spans multiple lines
4    and has code inside: let x = 42; */
5 let x = 10;
```

Here lines 3 and 4 are inside the comment. But if we use a line by line regex, it does not see the opening `/*` on those lines, so it treats them as real code. It will try to tokenize words like `spans`, `multiple`, `let`, `x`, `=`, `42` as Jack tokens. This will either cause a parse error or produce wrong output.

How state machine fixes this

The state machine reads one character at a time and remembers which state it is in. The four states are:

1. **IN_CODE** – normal code, keep all characters
2. **IN_LINE_COMMENT** – inside `//`, skip until newline
3. **IN_BLOCK_COMMENT** – inside `/* */`, skip until `*/`
4. **IN_STRING** – inside a string, keep everything

When it sees `/*` it goes to `IN_BLOCK_COMMENT` and stays there across as many lines as needed until it finds `*/`. This is why it works correctly for multi-line comments.

Q2 : Symbol Table inside `Conv.conv2d()`

Note: The assignment uses both `convolve` in Section 2.2 and `conv2d` in the Section 2.1 skeleton. We used `conv2d` to match the skeleton.

When the parser enters `conv2d`, there are two scopes. The class scope has all the class level variables. The subroutine scope has all the local variables of `conv2d`.

Class scope

Name	Type	Kind	Index
`ArrayFilter`	`Array`	STATIC	0
`stride`	`int`	STATIC	1
`ArrayInput`	`Array`	FIELD	0
`ArrayOutput`	`Array`	FIELD	1
`ArraySize`	`int`	FIELD	2

`STATIC` maps to the `static` segment in VM. `FIELD` maps to the `this` segment in VM.

Subroutine scope

Since `conv2d` is a method, the compiler adds `this` as `ARG 0` automatically. All `var` variables map to `local` in VM.

Name	Type	Kind	Index
`this`	`Conv`	ARG	0
`outDim`	`int`	VAR	0
`outRow`	`int`	VAR	1
`outCol`	`int`	VAR	2
`inRow`	`int`	VAR	3
`inCol`	`int`	VAR	4
`fRow`	`int`	VAR	5
`fCol`	`int`	VAR	6
`sum`	`int`	VAR	7
`k`	`int`	VAR	8
`inputIdx`	`int`	VAR	9
`filterIdx`	`int`	VAR	10

At the line where `sum` is updated, the compiler looks up `sum` and gets `local 7`, `ArrayInput` gives `this 0`, and `ArrayFilter` gives `static 0`. The subroutine scope is checked first, then the class scope.

Test Case: Input and Filter Used

5×5 Input Matrix

We used values 1 to 25 filled row by row:

$$\text{Input} = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{pmatrix}$$

We picked these values because each row is exactly 5 more than the row above it. This makes it easy to calculate the expected output by hand before running the program.

3×3 Filter

We used a horizontal edge detection filter:

$$\text{Filter} = \begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ +1 & +1 & +1 \end{pmatrix}$$

The top row is -1, middle row is 0, bottom row is $+1$. It computes the difference between the bottom and top rows of each window.

Expected output

For the top left window (rows 0, 1, 2 and cols 0, 1, 2):

$$(-1)(1+2+3) + (0)(6+7+8) + (+1)(11+12+13) = -6+0+36 = 30$$

Every window gives the same result because the row gap is always 5. So the output is:

$$\text{Output} = \begin{pmatrix} 30 & 30 & 30 \\ 30 & 30 & 30 \\ 30 & 30 & 30 \end{pmatrix}$$

This was verified on the NAND2Tetris Web IDE VM Emulator and the screen showed exactly these values (screenshot included in submission).

9×9 Input Matrix (Extra Credit)

For the 9x9 test case we filled the input with values 1 to 81, row by row:

$$\text{Input} = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 10 & 11 & 12 & 13 & 14 & 15 & 16 & 17 & 18 \\ 19 & 20 & 21 & 22 & 23 & 24 & 25 & 26 & 27 \\ 28 & 29 & 30 & 31 & 32 & 33 & 34 & 35 & 36 \\ 37 & 38 & 39 & 40 & 41 & 42 & 43 & 44 & 45 \\ 46 & 47 & 48 & 49 & 50 & 51 & 52 & 53 & 54 \\ 55 & 56 & 57 & 58 & 59 & 60 & 61 & 62 & 63 \\ 64 & 65 & 66 & 67 & 68 & 69 & 70 & 71 & 72 \\ 73 & 74 & 75 & 76 & 77 & 78 & 79 & 80 & 81 \end{pmatrix}$$

We used the same filter as before. The output dimension with stride 1 is:

$$O = \left\lfloor \frac{9-3}{1} \right\rfloor + 1 = 7$$

So the output is 7×7. Each row in this input is 9 more than the row above it. So the filter output for every window is:

$$(-1)(3 \text{ values}) + (0)(3 \text{ values}) + (+1)(3 \text{ values}) = 3 \times 9 + 3 \times 9 = 54$$

$$\text{Output} = \begin{pmatrix} 54 & 54 & 54 & 54 & 54 & 54 & 54 \\ 54 & 54 & 54 & 54 & 54 & 54 & 54 \\ 54 & 54 & 54 & 54 & 54 & 54 & 54 \\ 54 & 54 & 54 & 54 & 54 & 54 & 54 \\ 54 & 54 & 54 & 54 & 54 & 54 & 54 \\ 54 & 54 & 54 & 54 & 54 & 54 & 54 \\ 54 & 54 & 54 & 54 & 54 & 54 & 54 \end{pmatrix}$$

The emulator showed all 54s on screen which confirmed it is correct.

Q3 : VM code for `let ArrayOutput[k] = sum;`

Why it is not simple

In Jack VM, you cannot directly write to an address that is calculated at runtime. You can only write to fixed segments like `local 3` or `this 1`. But here the index `k` changes every loop iteration, so we need another way.

The pointer 1 and that 0 method

The VM has a special segment called `that`. If you write an address into `pointer 1`, then `that 0` points to that address in RAM. So we can point `that` wherever we want and then write there.

VM code (for k=4, sum=42)

```
// find the target address
push this 1       // base address of ArrayOutput (FIELD 1)
push local 8      // value of k = 4
add               // base + 4 is now on stack

// get the value to write
push local 7      // sum = 42

// write using pointer 1 and that 0
pop temp 0        // save 42 in temp 0
pop pointer 1     // THAT = base + 4
push temp 0       // bring 42 back
pop that 0        // RAM[THAT] = 42
```

If `ArrayOutput` starts at RAM address 2100, then `THAT` becomes 2104 and `pop that 0` writes 42 into RAM[2104], which is `ArrayOutput[4]`. This is correct.

We save to `temp 0` first because both the address and the value are on the stack at the same time. We need to pop them in the right order without losing either one.

Q4 : Stride Analysis

Output size for 9×9 input, $F = 3$, $P = 0$, $S = 2$

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1 = \left\lfloor \frac{9 - 3}{2} \right\rfloor + 1 = 3 + 1 = 4$$

Output is 4×4 which has 16 values.

Does stride change the number of VM instructions?

The loop in `conv2d` looks like this:

```
for outRow in 0..O-1:       // O times
  for outCol in 0..O-1:     // O times
    for fRow in 0..2:        // 3 times always
      for fCol in 0..2:      // 3 times always
        // fixed number of instructions
```

Total instructions $= O^2 \times 9 \times c$ where c is a fixed number.
Since $O \approx N/S$ for large N:

$$T \approx \left(\frac{N}{S}\right)^2 \times 9c = \frac{9c}{S^2} \cdot N^2$$

So both $S = 1$ and $S = 2$ are $O(N^2)$. The complexity class does not change. But the actual number of instructions is about 4 times less for $S = 2$ compared to $S = 1$, because the output grid is 4 times smaller.

Extra Credit: Bug I Found While Building the Parser

The bug was in how I handled `let a[expr1] = expr2;`.

In my first version, the code computed `base + expr1` first, then evaluated `expr2`. After both, the stack had the address on the bottom and the value on top. Then I did:

```
1 pop pointer 1     // set THAT to address
2 pop that 0        // write value
```

But this is wrong. `pop pointer 1` takes the top of the stack which is the value, not the address. So THAT gets set to the wrong thing and the value goes to the wrong place.

The fix is to save the value in `temp 0` first, then set THAT, then bring the value back and write it:

```
1 pop temp 0        // save value
2 pop pointer 1     // now THAT = correct address
3 push temp 0       // bring value back
4 pop that 0        // write to correct place
```

I found this bug by stepping through the VM emulator one instruction at a time and checking where the value was being written in RAM.